

# Smart Contract Security Audit Report

Generated by DeFi Price Manipulation Analyzer · v3.4

**Overall Risk Level: Critical**

Contract Name: **TesseraSwap**

Blockchain: **ethereum**

Analysis Date: **7/30/2026, 2:18:28 AM**

Total Vulnerabilities: **11**

## 1. Executive Summary

**4**

Critical

**6**

High

**1**

Medium

**0**

Low

### Urgent Recommendations

- ⚠ Instant engine replacement by owner enables protocol-wide fund theft: Implement a timelock (e.g., 48h delay) and require multi-signature approval for engine changes. Consider a two-step ownership transfer with acceptance by the new engine address.
- ⚠ Swap pricing relies exclusively on an external engine with no fallback: Integrate a trusted on-chain oracle (e.g., Chainlink) as a reference price, and enforce a deviation threshold against the engine's quoted price. Alternatively, implement a TWAP directly in the swap contract or require multi-source aggregation.
- ⚠ Swap pricing likely depends on manipulatable spot reserves: Replace the reliance on instantaneous spot reserves with a time-weighted average price (TWAP) from the AMM, or integrate a Chainlink oracle. Ensure the TWAP observation window is at least 30 minutes.

⚠ Reentrancy in tesseraSwapWithAllowances allows draining of treasury tokens: Add the nonReentrant modifier from OpenZeppelin's ReentrancyGuard to the tesseraSwapWithAllowances function, or follow the Checks-Effects-Interactions pattern by transferring tokenIn before tokenOut.

## 2. Project Overview

|                  |                       |
|------------------|-----------------------|
| Contract Name    | TesseraSwap           |
| Contract Address | TesseraSwap.sol       |
| Blockchain       | ethereum              |
| Analysis Date    | 7/30/2026, 2:18:28 AM |
| Quality Score    | C                     |

## 3. Vulnerability Details

**Critical**

AC-01

## Instant engine replacement by owner enables protocol-wide fund theft

The `changeTesseraEngine` function allows the single owner address to instantly replace the core price/exchange engine contract. A malicious owner or compromised key can redirect all swaps to a malicious engine, draining user and treasury funds in one transaction.

Code Location: TesseraSwap.sol | L84-L87 | changeTesseraEngine()

```
function changeTesseraEngine(address newTesseraEngine) external {  
    require(msg.sender == tesseraOwner, "ACE");  
    tesseraEngine = ITesseraEngine(newTesseraEngine);  
}
```

**Attack Vector:** 1. Malicious owner calls changeTesseraEngine with address of a fake engine. 2. All subsequent swap calls via tesseraSwapWithAllowances or tesseraSwapWithCallback will use the fake engine, which can manipulate amounts and steal tokens. 3. For example, fake engine returns a tiny amountOut but requests large amountIn, and the treasury/recipient lose funds.

**Impact Analysis:** Complete loss of all assets handled by the swap contract, affecting all users who have approved allowances or funds routed through the contract.

Historical Cases:

CASE-001 - CUT Token Price Protection Manipulation (Similarity: 72%)

**Fix Recommendation:** Implement a timelock (e.g., 48h delay) and require multi-signature approval for engine changes. Consider a two-step ownership transfer with acceptance by the new engine address.

**High**

T0-01

## Swap functions lack deadline parameter exposing users to MEV and stale orders

Neither `tesseraSwapWithAllowances` nor `tesseraSwapWithCallback` includes a deadline parameter. Swaps can be held in the mempool and executed at any future time when market conditions may have drastically changed, causing slippage beyond what the user intended, or the user may have already expected the transaction to be cancelled.

Code Location: TesseraSwap.sol | L38-L52 | tesseraSwapWithAllowances()

```
function tesseraSwapWithAllowances(  
    address tokenIn,  
    address tokenOut,  
    int256 amountSpecified,  
    uint256 amountCheck,  
    address recipient,  
    bytes calldata swapData  
) external { ... }
```

**Attack Vector:** 1. User submits a swap transaction with a certain amountCheck. 2. The transaction remains pending due to low gas or MEV bots. 3. Over time, the engine's exchange rate changes unfavorably. 4. The transaction eventually executes, resulting in a worse rate than the user would have accepted, potentially leading to financial loss even though the amountCheck still passes (it may be the limit, not the desired outcome).

**Impact Analysis:** Users may suffer from frontrunning or stale execution, receiving less favorable rates or having their trade execute under unintended conditions, violating the expectation of timely execution.

**Fix Recommendation:** Add a `deadline` parameter to both swap functions and require `block.timestamp <= deadline`. Follow the pattern used in Uniswap V2 router.

**High**

AC-02

### Owner can drain all contract tokens via rescueTokens

The rescueTokens function allows the owner to transfer any token or ETH held by the contract to the treasury, with no restrictions on amount or token type. This includes tokens that are part of the swap mechanism, enabling the owner to rug pull all funds.

Code Location: TesseractSwap.sol | L94-L106 | rescueTokens()

```
function rescueTokens(address[] memory tokens, uint256[] memory amounts) external {
    require(msg.sender == tesseraOwner, "ACE");
    ...
    IERC20(tokenAddress).safeTransfer(tesseraTreasury, tokenAmount);
}
```

**Attack Vector:** 1. Owner (malicious or compromised) calls rescueTokens with all token addresses held by the contract. 2. All funds are immediately transferred to the treasury, which may also be controlled by the owner.

**Impact Analysis:** Total loss of all user and protocol funds held in the contract.

Historical Cases:

AC-002 - Compounder Rug Pull (Similarity: 85%)

**Fix Recommendation:** Restrict rescueTokens to only tokens accidentally sent (e.g., track deposits), or implement a governance/timelock process for such withdrawals.

**Critical**

CR-01

## Swap pricing relies exclusively on an external engine with no fallback

Both swap functions (`tesseraSwapWithAllowances` and `tesseraSwapWithCallback`) call `tesseraEngine.swapAmount` to obtain the `amountIn/amountOut`. There is no secondary price source or oracle; the engine's return values are trusted without any circuit breaker. If the engine is compromised (e.g., via a bug, manipulation attack, or malicious owner update), the swap contract will execute trades at completely manipulated prices.

Code Location: `TesseraSwap.sol` | L35-L36 | `tesseraSwapWithAllowances()`

```
(uint256 amountIn, uint256 amountOut) = tesseraEngine.swapAmount(tokenIn, tokenOut, amountSpecified, msg.sender, swapData);
```

**Attack Vector:** 1. Attacker exploits a vulnerability in `tesseraEngine` (or convinces owner to update it to a malicious one via AC-01). 2. The engine returns a highly skewed `amountOut` for a given `amountIn`. 3. Attacker calls `tesseraSwapWithAllowances` and drains the treasury by exchanging minimal `tokenIn` for large `tokenOut`.

**Impact Analysis:** Protocol insolvency; all liquidity can be drained in a single transaction.

Historical Cases:

CASE-001 - CUT token price manipulation via reserve ratio (Similarity: 60%)

**Fix Recommendation:** Integrate a trusted on-chain oracle (e.g., Chainlink) as a reference price, and enforce a deviation threshold against the engine's quoted price. Alternatively, implement a TWAP directly in the swap contract or require multi-source aggregation.

**High**

CR-04

## Engine return values are used without any consistency or deviation check

After calling `tesseraEngine.swapAmount`, the returned `amountIn/amountOut` are trustfully used to validate `amountCheck` and to execute transfers. No independent verification of the engine's output is performed. Any subtle manipulation of the engine's internal state (e.g., via flash-loan induced price distortion) will directly propagate to the swap contract.

Code Location: `TesseraSwap.sol` | L35-L43 | `tesseraSwapWithAllowances()`

```
(uint256 amountIn, uint256 amountOut) = tesseraEngine.swapAmount(...);  
require(...);  
IERC20(tokenOut).safeTransferFrom(tesseraTreasury, recipient, amountOut);  
IERC20(tokenIn).safeTransferFrom(msg.sender, tesseraTreasury, amountIn);
```

**Attack Vector:** 1. Attacker takes a flash loan to temporarily move the reserves of the underlying AMM used by `tesseraEngine`. 2. Calls `tesseraSwapWithAllowances`; the engine reports a favorable swap rate based on the manipulated reserves. 3. The contract executes the swap, sending more `tokenOut` than fair value. 4. Attacker repays the flash loan and keeps the profit.

**Impact Analysis:** Loss of funds for the treasury and liquidity providers of the underlying engine.

Historical Cases:

[CASE-004 - LavaLending price manipulation \(Similarity: 70%\)](#)

**Fix Recommendation:** Before trusting the engine's output, compare it against a Chainlink price feed or a time-weighted average price. Revert if the deviation exceeds a safe threshold (e.g., 5%).

High

T0-03

### Missing reentrancy guard on tesseraSwapWithAllowances

The tesseraSwapWithAllowances function performs an external call to the engine and then interacts with ERC20 tokens, but does not use the nonReentrant modifier (unlike tesseraSwapWithCallback). A malicious token contract could call back into this function, potentially exploiting the engine's state to drain funds.

Code Location: TesseraSwap.sol | L42-L42 | tesseraSwapWithAllowances()

```
IERC20(tokenOut).safeTransferFrom(tesseraTreasury, recipient, amountOut);
```

**Attack Vector:** 1. Attacker crafts a malicious ERC20 token that triggers a callback in its `safeTransferFrom` function. 2. They call tesseraSwapWithAllowances with this token. 3. During the transfer, the callback re-enters the function, performing a second swap before the first completes, potentially manipulating the engine's internal state to extract excess value.

**Impact Analysis:** Possible theft of funds from the engine if it has reentrancy vulnerabilities, and user fund loss.

Historical Cases:

[CASE-008 - P719 Price Calculation Flaw \(Similarity: 30%\)](#)

**Fix Recommendation:** Add the `nonReentrant` modifier to `tesseraSwapWithAllowances`.



**Critical**

OD-01

## Swap pricing likely depends on manipulatable spot reserves

The `tesseraEngine.swapAmount` function (implementation not visible) presumably uses the current spot reserves of an underlying AMM pool to compute the exchange amounts. The `TesseraSwap` contract directly adopts these amounts without any time-weighted averaging, making every swap vulnerable to single-transaction price manipulation via flash loans.

Code Location: `TesseraSwap.sol` | L35-L36 | `tesseraSwapWithAllowances()`

```
(uint256 amountIn, uint256 amountOut) = tesseraEngine.swapAmount(tokenIn, tokenOut, amountSpecified, msg.sender, swapData);
```

**Attack Vector:** 1. Attacker obtains a large flash loan in `tokenIn`. 2. Swaps `tokenIn` for `tokenOut` on the underlying AMM, massively depleting `tokenOut` reserves. 3. Calls `TesseraSwap` with a large `amountSpecified`; the engine returns an artificially high `amountOut` based on the manipulated reserves. 4. Attacker receives the inflated `amountOut` from `TesseraSwap`'s treasury, then repays the flash loan.

**Impact Analysis:** Treasury drained; total loss of all liquidity.

Historical Cases:

[CASE-004 - LavaLending price manipulation \(Similarity: 85%\)](#)

**Fix Recommendation:** Replace the reliance on instantaneous spot reserves with a time-weighted average price (TWAP) from the AMM, or integrate a Chainlink oracle. Ensure the TWAP observation window is at least 30 minutes.

**Critical**

TO-03

## Reentrancy in tesseraSwapWithAllowances allows draining of treasury tokens

The tesseraSwapWithAllowances function transfers tokenOut to the recipient before collecting tokenIn from the sender, without a reentrancy guard. A malicious token (or a callback during safeTransferFrom) can re-enter the function and execute the swap again, draining tokenOut before any tokenIn is received.

Code Location: TesseraSwap.sol | L39-L47 | tesseraSwapWithAllowances()

```
IERC20(tokenOut).safeTransferFrom(tesseraTreasury, recipient, amountOut);  
IERC20(tokenIn).safeTransferFrom(msg.sender, tesseraTreasury, amountIn);
```

**Attack Vector:** 1. Attacker deploys a token with a transfer hook that calls tesseraSwapWithAllowances again. 2. Attacker calls tesseraSwapWithAllowances using this token as tokenOut. 3. During the safeTransferFrom of tokenOut, the hook re-enters tesseraSwapWithAllowances. 4. The re-entered call passes the same amountCheck (since tokenIn still not received) and transfers more tokenOut. 5. The attacker repeats until the treasury is drained.

**Impact Analysis:** Treasury can be completely drained by repeatedly stealing tokenOut before any tokenIn is collected.

Historical Cases:

[REENT-001 - The DAO Reentrancy \(Similarity: 85%\)](#)

**Fix Recommendation:** Add the nonReentrant modifier from OpenZeppelin's ReentrancyGuard to the tesseraSwapWithAllowances function, or follow the Checks-Effects-Interactions pattern by transferring tokenIn before tokenOut.

High

CR-01

### Swap pricing entirely depends on a single TesseractEngine without fallback

All swap functions call `tesseractEngine.swapAmount()` to determine the exchange amounts. There is no alternative source or circuit breaker if the engine is compromised, manipulated, or malfunctioning. Any attack on the engine directly controls the swap outcome.

Code Location: `TesseractSwap.sol` | L40-L40 | `tesseractSwapWithAllowances()`

```
(uint256 amountIn, uint256 amountOut) = tesseractEngine.swapAmount(tokenIn, tokenOut, amountSpecified, msg.sender, swapData);
```

**Attack Vector:** If the TesseractEngine's internal state is manipulated (e.g., via flash loan), the `swapAmount` function will return a manipulated exchange rate, causing the swap to occur at a severely distorted price, resulting in losses for the user or the treasury.

**Impact Analysis:** Users can receive extremely poor rates; treasury can be drained by arbitrage due to incorrect pricing.

Historical Cases:

[CR-001 - bZx flash loan attack \(Similarity: 80%\)](#)

**Fix Recommendation:** Integrate a secondary oracle (e.g., Chainlink) to cross-check the engine's output. Implement a deviation threshold that pauses swaps if the engine's price deviates too much from the oracle.

High

AC-02

### Owner can change treasury address instantly

The changeTesseraTreasury function allows the owner to redirect where swap proceeds are sent without any timelock or multi-sig. This could be used to steal funds by pointing the treasury to an attacker-controlled address.

Code Location: TesseraSwap.sol | L86-L86 | changeTesseraTreasury()

```
tesseraTreasury = newTesseraTreasury;
```

**Attack Vector:** 1. Owner changes treasury to a malicious address. 2. On the next swap, tokens flow to the new treasury instead of the intended one, effectively stealing them.

**Impact Analysis:** Potential redirect of user funds to an attacker-controlled address.

**Fix Recommendation:** Add a timelock and multi-sig requirement for treasury address changes.

Medium

CR-04

### No validation of engine-provided swap amounts

The engine's `swapAmount` output is used directly without any on-chain sanity check against an independent price source, allowing a manipulated engine to return arbitrary values.

Code Location: TesseraSwap.sol | L37-L38 | tesseraSwapWithAllowances()

```
(uint256 amountIn, uint256 amountOut) = tesseraEngine.swapAmount(tokenIn, tokenOut, amountSpecified, msg.sender, swapData);
```

**Attack Vector:** If the engine's internal state is manipulated (e.g., via flash loan), it can report a favorable rate to the attacker, who then uses the swap to drain funds within the user's slippage tolerance.

**Impact Analysis:** Attackers can extract value by temporarily distorting the engine's pricing, causing large losses for the treasury if slippage checks are insufficient.

Historical Cases:

[CASE-004 - LavaLending price manipulation \(Similarity: 70%\)](#)

**Fix Recommendation:** Add a price deviation check: compare the engine's amountOut against a Chainlink TWAP and revert if the discrepancy exceeds a threshold (e.g., 10%).

## 4. Risk Matrix

### 4.1 By Severity Level

| Severity Level | Count | Percentage |
|----------------|-------|------------|
| Critical       | 4     | 36.4%      |
| High           | 6     | 54.5%      |

|        |   |      |
|--------|---|------|
| Medium | 1 | 9.1% |
| Low    | 0 | 0.0% |

4.2 By Vulnerability Pattern

| Pattern                                                                              | Count | Highest Severity |
|--------------------------------------------------------------------------------------|-------|------------------|
| Oracle Address Update Without Timelock/Multi-sig                                     | 1     | Critical         |
| Missing Deadline / Protection Window                                                 | 1     | High             |
| Economic Parameters Adjustable by Privileged Role                                    | 2     | High             |
| Reliance on External Protocol / Bridge as Sole Price Source                          | 2     | Critical         |
| Unverified Cross-Protocol Price Dependency                                           | 2     | High             |
| State Changed Multiple Times in One Transaction (Reentrancy-Type)                    | 1     | High             |
| Spot Price Directly Used as Pricing Basis                                            | 1     | Critical         |
| State Changed Multiple Times in One Transaction (Reentrancy-Type Price Manipulation) | 1     | Critical         |

5. Code Quality Assessment

|               |   |
|---------------|---|
| Quality Score | C |
|---------------|---|

Specific Issues

- Missing reentrancy guard on `tesseraSwapWithAllowances`
- Missing deadline parameter in swap functions
- Centralized control over critical engine address
- Unrestricted token rescue function gives owner full withdrawal ability
- Heavy reliance on a single external engine without fallback or validation
- No on-chain verification of engine-provided amounts

## 6. Conclusion & Recommendations

---

### Phased Remediation Plan

#### Immediate Fix (Before Deployment)

- Instant engine replacement by owner enables protocol-wide fund theft: Implement a timelock (e.g., 48h delay) and require multi-signature approval for engine changes. Consider a two-step ownership transfer with acceptance by the new engine address.
- Swap pricing relies exclusively on an external engine with no fallback: Integrate a trusted on-chain oracle (e.g., Chainlink) as a reference price, and enforce a deviation threshold against the engine's quoted price. Alternatively, implement a TWAP directly in the swap contract or require multi-source aggregation.
- Swap pricing likely depends on manipulatable spot reserves: Replace the reliance on instantaneous spot reserves with a time-weighted average price (TWAP) from the AMM, or integrate a Chainlink oracle. Ensure the TWAP observation window is at least 30 minutes.
- Reentrancy in `tesseraSwapWithAllowances` allows draining of treasury tokens: Add the `nonReentrant` modifier from OpenZeppelin's `ReentrancyGuard` to the `tesseraSwapWithAllowances` function, or follow the Checks-Effects-Interactions pattern by transferring `tokenIn` before `tokenOut`.

#### Short-term Fix (7-30 days)

- Swap functions lack deadline parameter exposing users to MEV and stale orders: Add a `deadline` parameter to both swap functions and require `block.timestamp <= deadline`. Follow the pattern used in Uniswap V2 router.
- Owner can drain all contract tokens via `rescueTokens`: Restrict `rescueTokens` to only tokens accidentally sent (e.g., track deposits), or implement a governance/timelock process for such withdrawals.

- Engine return values are used without any consistency or deviation check: Before trusting the engine's output, compare it against a Chainlink price feed or a time-weighted average price. Revert if the deviation exceeds a safe threshold (e.g., 5%).
- Missing reentrancy guard on `tesseraSwapWithAllowances`: Add the `nonReentrant` modifier to `tesseraSwapWithAllowances`.
- Swap pricing entirely depends on a single TesseractEngine without fallback: Integrate a secondary oracle (e.g., Chainlink) to cross-check the engine's output. Implement a deviation threshold that pauses swaps if the engine's price deviates too much from the oracle.
- Owner can change treasury address instantly: Add a timelock and multi-sig requirement for treasury address changes.

Long-term Improvement (Future versions)

- No validation of engine-provided swap amounts: Add a price deviation check: compare the engine's `amountOut` against a Chainlink TWAP and revert if the discrepancy exceeds a threshold (e.g., 10%).

Risk Rating Criteria

| Severity Level | Definition                              | Impact                    | Fix Timeline   |
|----------------|-----------------------------------------|---------------------------|----------------|
| Critical       | Can directly cause major fund loss      | User funds at direct risk | Immediate      |
| High           | Exploitable under specific conditions   | Major security risk       | Within 7 days  |
| Medium         | Requires specific conditions to exploit | Potential risk            | Within 30 days |
| Low            | Theoretical risk or best practice issue | Limited impact            | Recommended    |



This report is generated by AI-assisted analysis and should be reviewed by security professionals before making deployment decisions.